

# P3618R0

## Allow attaching main to the global module

Published Proposal, 2025-02-10

**This version:**

<http://wg21.link/P3618R0>

**Author:**

[Michael Spencer](#) (Apple)

**Audience:**

EWG

**Project:**

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

### Abstract

This paper presents an alternative to [P3422R1] - Allow main function in named modules, allowing `main` to be defined in a module, but by removing special cases on `main` instead of adding them.

### Table of Contents

- 1 Effects of This Paper
- 2 Motivation
  - 2.1 Testing Modules
- 3 Why not P3422?
  - 3.1 Modular `main()`
  - 3.2 More Special Cases For `main`
- 4 Allowing Language Linkage on `main`
- 5 Compatibility
- 6 Wording
- References
  - Informative References

#### § 1. Effects of This Paper

| C++23                                                                                                   | P3422                                                                                                   | This paper                                                                                       |
|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <pre>module M; // attached to M void f(); // attached to the global module extern "C++" void g();</pre> | <pre>module M; // attached to M void f(); // attached to the global module extern "C++" void g();</pre> | <pre>module M; // attached to M void f(); // attached to the global extern "C++" void g();</pre> |

```
// ill-formed
int main() {}
// ill-formed
extern "C++" int main() {}
```

```
// attached to the global module
int main() {}
// ill-formed
extern "C++" int main() {}
```

```
// ill-formed
int main() {}
// attached to the global
extern "C++" int main() {}
```

## § 2. Motivation

There is a reasonable desire to define `main` as part of a module. The most often cited reason is to support unit testing of non-exported entities of a module.

### § 2.1. Testing Modules

To be able to test non-exported entities of a module the test function must be a member of that module. One possible way to do this would be similar to the following:

```
module M;

extern "C++" int main() {
    // test non-exported entities in M here.
}
```

Here the TU containing this is an implementation unit and does not need to be part of every program using `M`. However, today this is ill-formed.

## § 3. Why not P3422?

The way P3422 solves this problem is to implicitly attach `main` defined in the purview of a named module to the global module instead. This does solve the problem, but it has two downsides.

### § 3.1. Modular `main()`

[P1203R0] - Modular `main()` is a paper that was presented in 2018 that proposed allowing `main` to be attached to a module, thus allowing one per module. The idea was that this would allow having an implementation defined way to select which `main` is invoked when a program has these, like many other modern languages. This was also motivated by the same reasons as P3422.

There was reasonable interest at the time in EWGI that modular `main` in some form was something they were interested in pursuing. The authors of P1203 are no longer pursuing this paper, but P3422 would largely block off this path.

P3422 does discuss P1203, but this is mostly around one way to select a `main`. Other languages allow specifically selecting which entry point to use by name, such as `M.main`, which does not have the problems mentioned in the paper. This paper does not propose modular `main`, just that we should keep that space open.

### § 3.2. More Special Cases For `main`

In C++23 the idiomatic way to attach a declaration in a module to the global module is to use `extern "C++"`. This was specifically added to modules to provide a way to move declarations into a module without breaking ABI. This is exactly what we want to do here with `main`.

Today having a linkage declaration on `main` is ill-formed, and has been since C++98. However, the primary reason this was prohibited is that it really had no meaning, as the linkage of `main` for ABI purposes is defined by the platform, and can't be changed. Now that `extern "C++"` has another meaning, I believe that it's fine to apply this to `main`. Implementations will still use whatever ABI is required by the platform, but will also attach it to the global module.

Making this change removes a special case from `main`, and avoids adding another special case. This makes the language more consistent.

## § 4. Allowing Language Linkage on `main`

Currently the linkage of `main` is implementation-defined and the language linkage is C++ (because the default language linkage of all functions is C++). This paper does not change that, it just allows restating that the language linkage is C++.

A language linkage specification does also add `extern` to the declaration; however, this is already allowed and is ignored because [basic.start.main]/3 says that the linkage is implementation-defined.

I do not believe there is any specification issue with allowing `extern "C++"`, and there is no implementation issue. Clang and GCC already allow linkage specifications on `main` with a warning even under `-pendantic`.

## § 5. Compatibility

From the standard's perspective this paper just makes previously ill-formed code well-formed.

In Clang and GCC this will just remove a warning as they already do the right thing. Implementations should make `main` without `extern "C++"` ill-formed, as the standard currently requires.

## § 6. Wording

Modify [basic.start.main]/3 as follows:

The function `main` shall not be named by an expression. The linkage ([basic.link]) of `main` is implementation-defined. A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, `constexpr`, or `constexpr` is ill-formed. The function `main` shall not be a coroutine ([dcl.fct.def.coroutine]). The `main` function shall not be declared with a *linkage-specification* ([dcl.link]) other than "C++". A program that declares

- a variable `main` that belongs to the global scope, or
- a function `main` that belongs to the global scope and is attached to a named module, or
- a function template `main` that belongs to the global scope, or
- an entity named `main` with C language linkage (in any namespace)

is ill-formed. The name `main` is not otherwise reserved.

[Example 1: Member functions, classes, and enumerations can be called `main`, as can entities in other namespaces. — end example]

## § References

### § Informative References

#### [P1203R0]

Boris Kolpackov, Richard Smith. *Modular main()*. 5 October 2018. URL: <https://wg21.link/p1203r0>

**[P3422R1]**

Chuanqi Xu. [\*Allow main function in named modules\*](https://wg21.link/p3422r1). 28 November 2024. URL: <https://wg21.link/p3422r1>